Secure Development and Critical Reflection of the Streeling University IMS

Student ID: 5653630, December 2025

The Cyber Context of Software Engineering (WM240), WMG, University of Warwick

---

Contents

1 Introduction **1**

2 Behaviour-Driven Development (BDD) **1**

3 System Design Phase **2**
3.1 Component Diagram (UML) 2
3.2 Architectural Component Analysis 3
3.3 Threat Modelling and Trust Boundary Analysis 3
3.4 Least privilege 3
3.5 Secure defaults 3
3.6 Defence in depth 4

4 Implementation **5**
4.1 Search and Filter 5
4.2 Loan and Inventory Management Workflow 6
4.3 Non-Functional Security Controls 8

5 Development & Challenges **10**
5.1 Solution 10
5.2 Testing 11

6 Critical Reflection and SSCoP Alignment **12**
6.1 Mapping to SSCoP Principles 12
6.2 Evaluation 13

7 Conclusion **13**

References **14**

Appendices **15**
A Established System Requirements 15

B Low-Level Architectural Diagram 16
C.1 Test 1 and 4: Student reserves available book 17
C.2 Test 2: Duplicate reservation by same user 19
C.3 Test 3: Reservation for unavailable book 19
C.4 Test 5: Approval attempt with existing active loan 20
C.5 Test 6: Librarian denies reservation 21
C.6 Test 7: User returns active loan 22
C.7 Test 8: Unauthenticated access to protected action 23
C.8 Test 9: Unauthenticated access to protected action 23
C.9 Test 10: Multiple failed login attempts within a short time window . . . 23
E Codebase Access Information 25

1 Introduction

This report outlines the implementation of an Inventory Management System (IMS) for Streeling University Library using a security-by-design approach, in order to ensure core functionalities are met without the compromise of security controls.

2 Behaviour-Driven Development (BDD)

This section outlines 6 core requirements set using the BDD format, including 3 functional and 3 non-functional requirements, with an additional 3 pre-established requirements defined in Appendix A.

Scenario: Brute-force Protection (Non-Functional)
Given a user is on the IMS login page,
When they submit more than five failed login attempts within one minute,
Then the system must block the source IP via rate-limiting,
And log the resource exhaustion attempt as a high-priority security event.

Scenario: Strict Input Sanitisation (Non-Functional)
Given a librarian is authenticated and performing a search query,
When they input data into the search or filter fields,
Then the system must apply strict input validation and parametrisation,
And reject inputs that fail to match the required secure schema (e.g., script tags).

Scenario: Session Management and Timeout (Non-Functional)
Given an authenticated user has an active session token,
When the user has been inactive for a predefined period (e.g., 30 minutes),
Then the system must automatically invalidate the JWT and log out the user,
And redirect the user to the login page to prevent unauthorised physical access.

Scenario: Book Loan Creation and Return Workflow (Functional)
Given a librarian is authenticated within the IMS,
When the librarian issues a loan for a selected book to a student,
Then the system must create a loan record with an associated due date and status,
And decrement the available inventory count for the corresponding book item.

Scenario: Prevention of Double-Loaning Unavailable Items (Functional)
Given a librarian is authenticated within the IMS,
When the librarian attempts to issue a loan for a book with zero available stock,
Then the system must deny the loan request,
And return a clear unavailability message to prevent inconsistent inventory state.

Scenario: Inventory Change History Visibility (Functional)
Given a library manager is authenticated within the IMS,
When the manager views the details of a specific inventory item,
Then the system must display a chronological history of inventory changes,
And include the timestamp and initiating user for each recorded action.

3 System Design Phase

3.1 Component Diagram (UML)

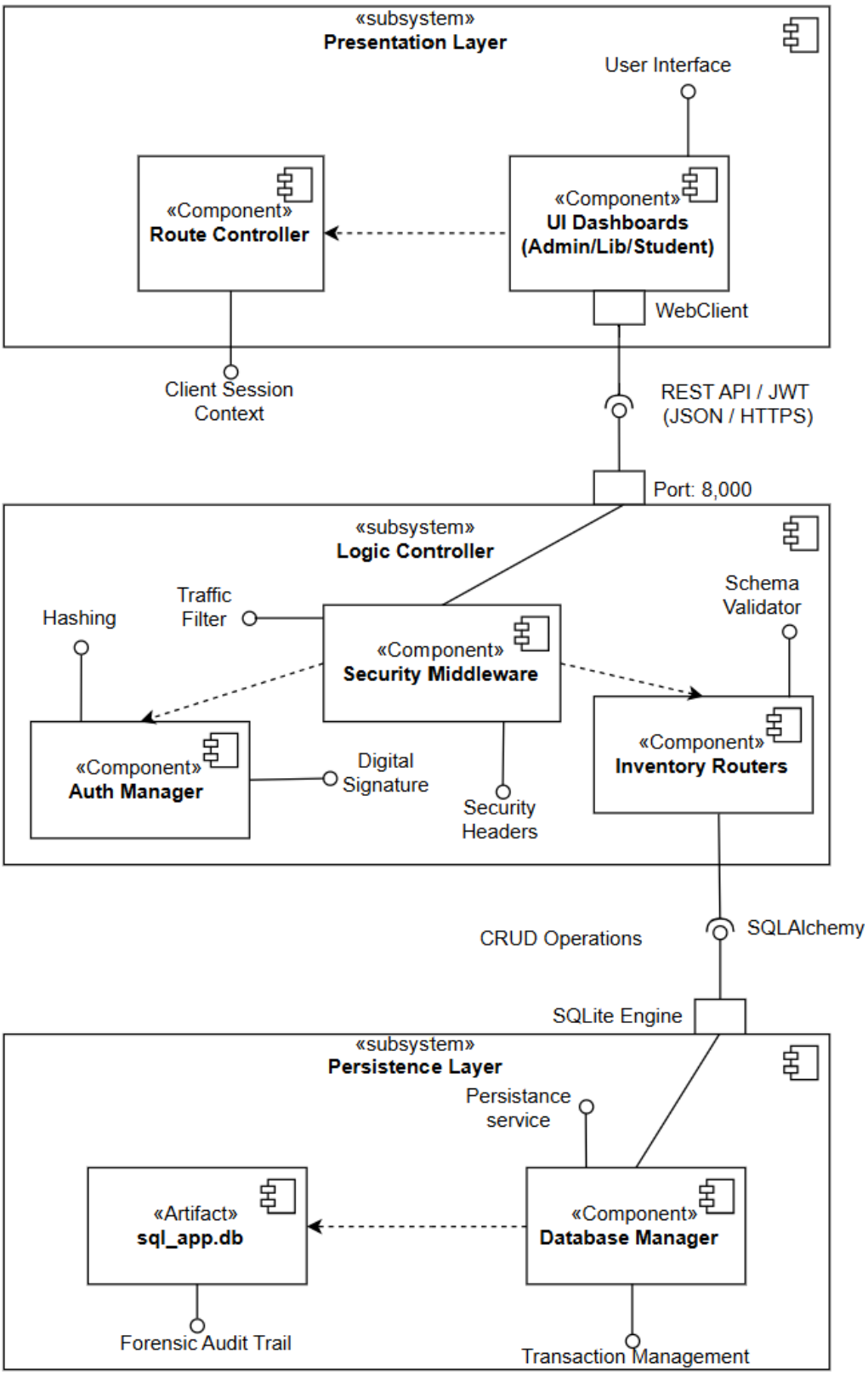


Figure 1: Three-Tier Component Diagram highlighting the IMS Security Plans.

3.2 Architectural Component Analysis

The component diagram above in Figure 1 showcases the application being divided into three logic layers, each encapsulated for a specific function. It starts with the presentation layer, which is responsible for providing the initial interface for the user based on their role, which is then stored as session context, enabling the user to interact with the IMS and forward requests. These requests are then validated by the logic controller, which acts as the backend of the system, where requests undergo strict verification and security controls before being hashed and allowed to interact with the backend data. Once these checks are satisfied, backend data storage is handled by the persistence layer, which uses an Object–Relational Mapper (ORM), in order to support parametrised queries and mitigate SQL injection attacks. Combined, these layers provide all necessary functionality needed by the user to perform all core inventory management tasks, without compromising security and data integrity.

3.3 Threat Modelling and Trust Boundary Analysis

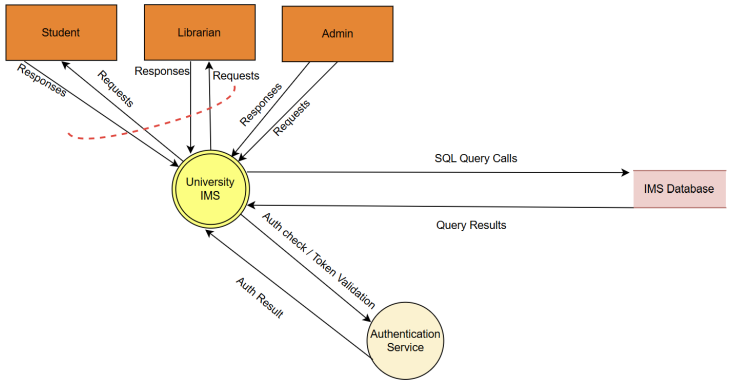


Figure 2: STRIDE-informed threat model for the University IMS

3.4 Least privilege

As shown in Figure 1, least privilege is maintained throughout the application by ensuring users only have the operational rights they need to complete their tasks. This can be seen in the UI Dashboard component, where only authenticated users are able to render their designated dashboard and maintain their user session context. This is because of a trust boundary that exists between the users and the University IMS as shown in Figure 2, ensuring privilege is controlled and routing to unauthorised endpoints is restricted.

3.5 Secure defaults

Secure defaults are displayed in Figure 1, inside the logic controller subsystem, where requests that are sent by the frontend (presentation layer) undergo strict validation checks before being able to access the backend data. This can be seen inside the Auth manager component, which is responsible for providing hashing and verifying identity through a digital signature, in order for requests to be queried to the database, as seen in Figure 2. This approach supports a controlled exposure of functionality, since only authenticated requests are allowed to proceed further into the system, preventing unauthorised access and ensuring the system fails are closed by default.

3.6 Defence in depth

Defence in depth is displayed all throughout the system by ensuring no security mechanism is trusted on its own. This can be seen in both Figure 1 and 2, where logic is split into three layers, each containing their own defence mechanism. For instance, for a user to perform SQL query calls to the database, they will first have to pass the trust boundary in order to gain access to the IMS, and will only be able to interact with the database once they have also passed through the authentication service. This structure enables the system to have no single point of failure, ensuring defence in depth is implemented through layered security controls.

4 Implementation

This section will focus on the implementation of the core functionality required for the application to be running. It presents a breakdown of the underlying code logic, followed by a set of test cases it will have to pass and their corresponding output, in order to ensure everything is fully functional and working.

4.1 Search and Filter

```
const filteredBooks = books.filter(book => {
    const matchesSearch =
        book.name.toLowerCase().includes(searchTerm.toLowerCase()) ||
        book.author.toLowerCase().includes(searchTerm.toLowerCase()) ||
        book.id.toLowerCase().includes(searchTerm.toLowerCase());

    let matchesFilter = true;
    if (filterField && filterValue) {
        let dataKey = filterField.toLowerCase();
        if (filterField === 'Book Name') dataKey = 'name';
        if (filterField === 'Release Date') dataKey = 'releaseDate';
        matchesFilter = String(book[dataKey]) === filterValue;
    }

    return matchesSearch && matchesFilter;
});
```

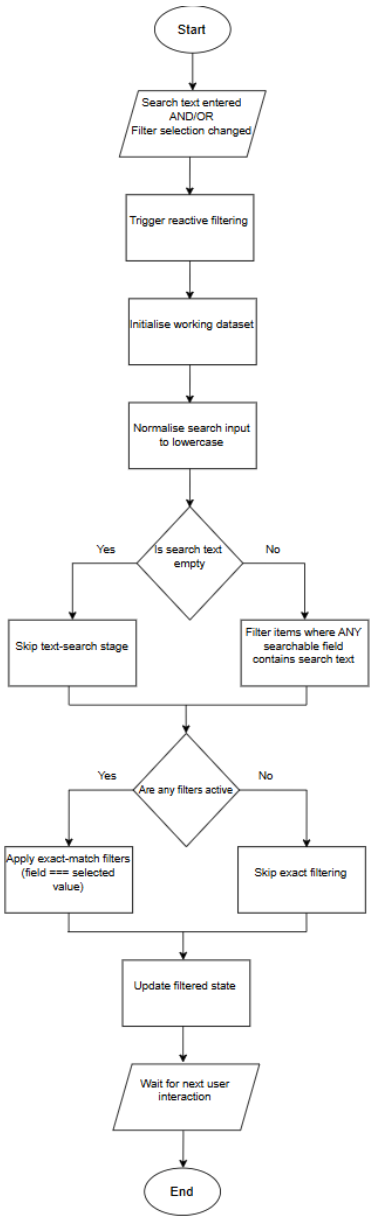
(a) Student Search Filter

```
useEffect(() => {
    let filtered = books;

    if (searchQuery) {
        const query = searchQuery.toLowerCase();
        filtered = filtered.filter(book =>
            book.name.toLowerCase().includes(query) ||
            book.author.toLowerCase().includes(query) ||
            book.id.toLowerCase().includes(query)
        );
    }

    if (filters.status) {
        filtered = filtered.filter(book => book.status === filters.status);
    }

    if (filters.category) {
        filtered = filtered.filter(book => book.category === filters.category);
    }

    setFilteredBooks(filtered);
}, [searchQuery, filters, books]);
```

(b) Librarian Search Filter

```
useEffect(() => {
    let res = logs;
    if (search) {
        const lower = search.toLowerCase();
        res = res.filter(l =>
            l.email.toLowerCase().includes(lower) ||
            l.action.toLowerCase().includes(lower) ||
            l.resource.toLowerCase().includes(lower)
        );
    }
    if (filters.Status) {
        res = res.filter(l => l.status === filters.Status);
    }
    setFilteredData(res);
}, [search, filters, logs]);
```

(c) Admin Search Filter



(d) Generalised Reactive Search and Filter Flow

Figure 3: Role-specific search/filter implementations (left) and the shared reactive filtering control flow (right).

Analysis: Figure 3 displays the search and filter as the primary interaction mechanism across all roles, supporting efficient data retrieval while maintaining system consistency. This is because each role has a distinct feature separating it from the rest. The student search filter (Figure 3a) only allows for basic high-visibility search to find books for reservation, while the librarian search filter (Figure 3b) takes this one step further by executing the filtered logic inside the `useEffect` callback, in order to combine search queries with explicit property filters. The admin search filter (Figure 3c) has the highest scope, as it follows the same reactive filtering control flow provided to the librarian role, while containing additional security features that enables it to isolate and map individual log properties, in order to monitor user activity and maintain data integrity.

4.2 Loan and Inventory Management Workflow

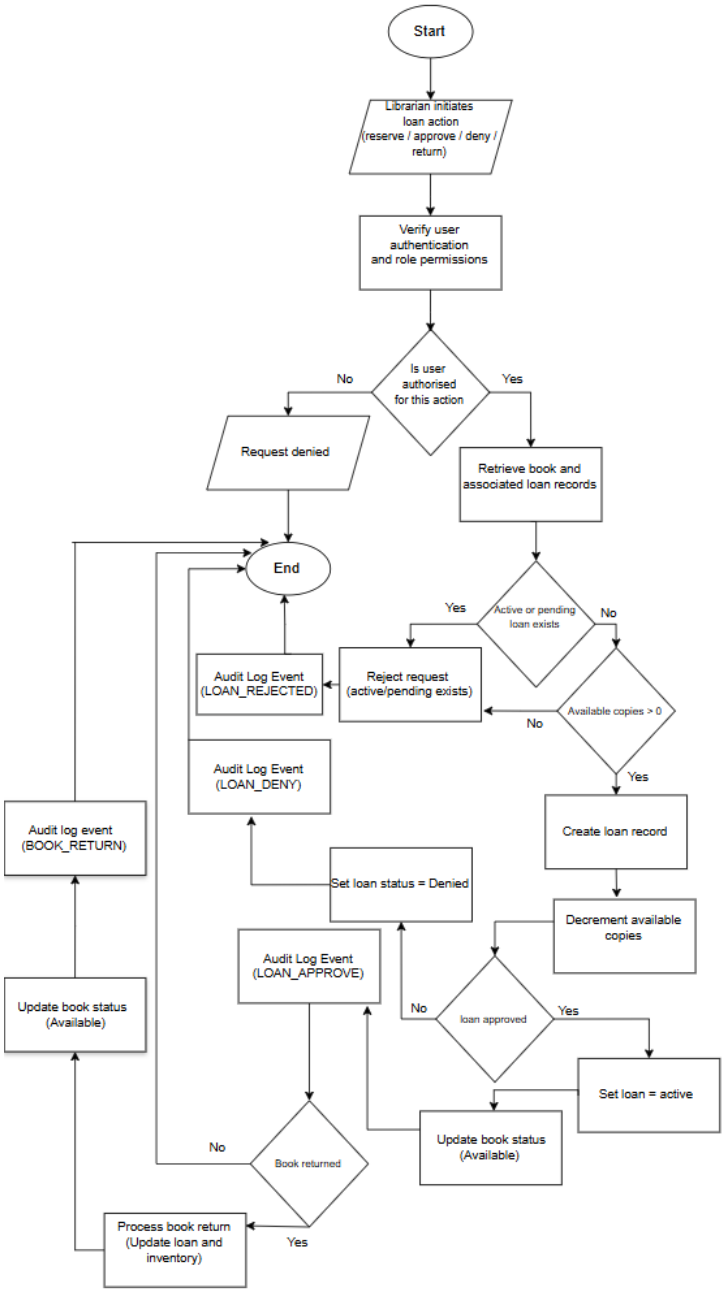


Figure 4: Unified loan and inventory management workflow.

```python
def reserve_book(loan_data: schemas.LoanCreate, db: Session = Depends(database.get_db), current_user: models.User = Depends(get_current_user)):
    existing_loan = db.query(models.Loan).filter(
        models.Loan.user_id == current_user.id,
        models.Loan.book_id == loan_data.book_id,
        models.Loan.status.in_(["Pending", "Active"])
    ).first()

    if existing_loan:
        raise HTTPException(status_code=400, detail="You already have an active/pending reservation for this book.")

    book = db.query(models.Book).filter(models.Book.id == loan_data.book_id).first()
    if not book:
        raise HTTPException(status_code=404, detail="Book not found")

    if book.available_copies <= 0:
        raise HTTPException(status_code=400, detail="No copies available for reservation")

    new_loan = models.Loan(
        user_id=current_user.id,
        username=current_user.username,
        book_id=loan_data.book_id,
        book_name=loan_data.book_name,
        status="Pending",
        borrow_date=datetime.now().strftime("%Y-%m-%d")
    )
    book.available_copies -= 1

    db.add(new_loan)
    db.commit()
    db.refresh(new_loan)
    audit.log_event(db, current_user.role, current_user.username, "BOOK_RESERVE", f"book:{loan_data.book_id}", "success")
    return new_loan
```

Figure 5: Loan Reservation and Validation

```python
def approve_loan(loan_id: int, loan_update: schemas.LoanUpdate, db: Session = Depends(database.get_db), current_user: models.User = Depends(allow_staff)):
    loan = db.query(models.Loan).filter(models.Loan.id == loan_id).first()
    if not loan:
        raise HTTPException(status_code=404, detail="Loan not found")

    loan.status = "Active"
    loan.due_date = loan_update.due_date

    book = db.query(models.Book).filter(models.Book.id == loan.book_id).first()
    if book and book.available_copies == 0:
        book.status = "Checked Out"

    db.commit()
    audit.log_event(db, current_user.role, current_user.username, "LOAN_APPROVE", f"loan:{loan_id} ({loan.book_name})", "success")
    return {"status": "success", "message": "Loan approved"}
```

Figure 6: Loan Approval Workflow

```python
def deny_loan(loan_id: int, loan_update: schemas.LoanUpdate, db: Session = Depends(database.get_db), current_user: models.User = Depends(allow_staff)):
    loan = db.query(models.Loan).filter(models.Loan.id == loan_id).first()
    if not loan:
        raise HTTPException(status_code=404, detail="Loan not found")

    book = db.query(models.Book).filter(models.Book.id == loan.book_id).first()
    if book:
        book.available_copies += 1
        book.status = "Available"

    loan.status = "Denied"
    loan.denial_reason = loan_update.denial_reason
    db.commit()
    audit.log_event(db, current_user.role, current_user.username, "LOAN_DENY", f"loan:{loan_id} ({loan.book_name})", "success")
    return {"status": "success", "message": "Loan denied"}
```

Figure 7: Loan Denial and Inventory Restoration

Analysis: Figure 4 presents the loan and inventory management workflow, where the process is initiated by a student making a request to reserve a book. This request is then carried out by the `reserve_book` operation (Figure 5), which is responsible for ensuring the requested book item is available in stock and was not already requested before by the same user. Once initial screening is passed, the process carries over to the next stage, where the `approve_loan` function makes sure the user is not trying to borrow a book they have already borrowed before and not returned. In cases where a request doesn't fit the required criteria, the `deny_loan` (Figure 7) function is called to allow only authorised users to deny requests and provide a reason for it. This methodology guarantees loan operations run both smoothly and reliably by making sure requests are consistently validated by authorised users, preserving inventory system integrity across all stages of the loans lifecycle.

4.3 Non-Functional Security Controls

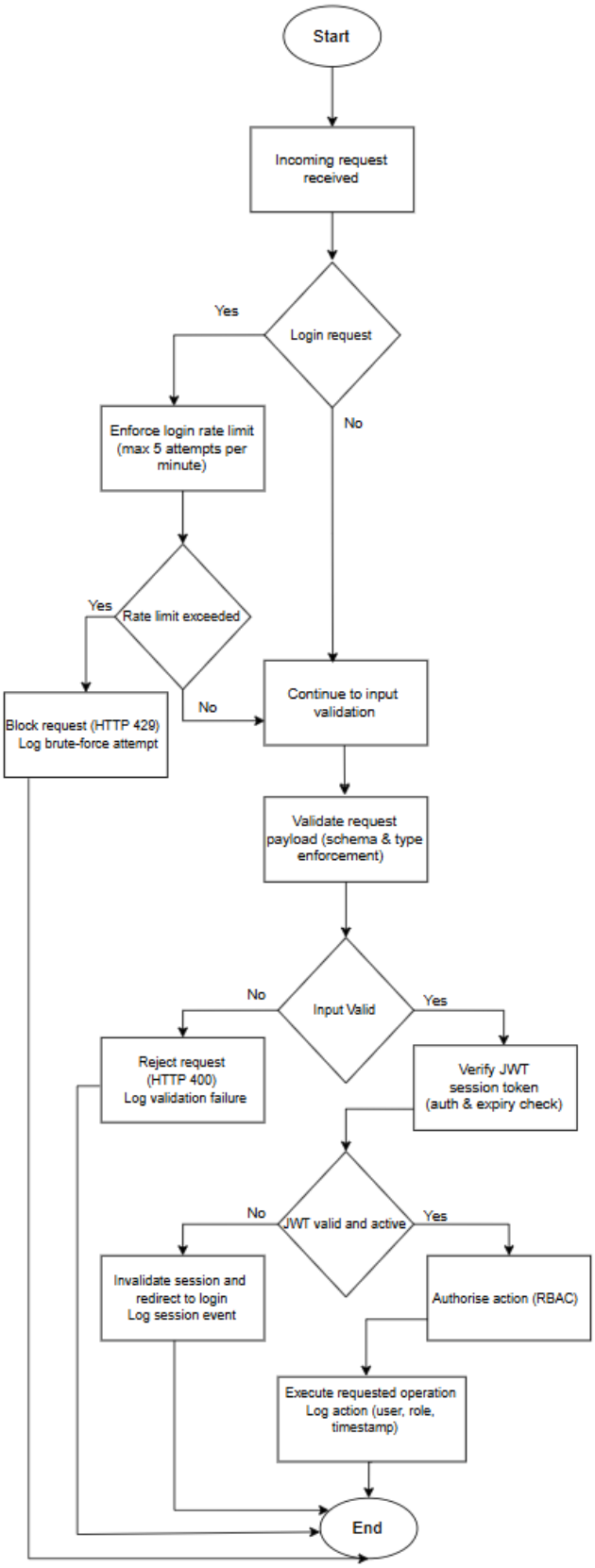


Figure 8: Unified Security Control Flow

```python
@router.post("/login")
@limiter.limit("5/minute")
def login(request: Request, user: schemas.UserLogin, db: Session = Depends(database.get_db)):
    username_lower = user.username.lower()
    db_user = db.query(models.User).filter(models.User.username == username_lower).first()

    if db_user and auth_utils.verify_password(user.password, db_user.password):
        if db_user.is_locked:
            raise HTTPException(status_code=403, detail="Account is locked. Please contact an administrator.")

        access_token = auth_utils.create_access_token(
            data={"sub": db_user.username, "role": db_user.role}
        )

        audit.log_event(db, db_user.role, db_user.username, "LOGIN", "auth:-", "success")

        return {
            "status": "success",
            "access_token": access_token,
            "token_type": "bearer",
            "role": db_user.role,
            "username": db_user.username
        }

    audit.log_event(db, "Visitor", user.username, "LOGIN_FAIL", "auth:-", "fail")
    raise HTTPException(status_code=401, detail="Invalid credentials")
```

Figure 9: Rate-Limited Login Endpoint and Audit Logging

```python
def get_password_hash(password: str) -> str:
    password_bytes = password.encode('utf-8')
    salt = bcrypt.gensalt()
    hashed = bcrypt.hashpw(password_bytes, salt)
    return hashed.decode('utf-8')

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None):
    to_encode = data.copy()
    if expires_delta:
        expire = datetime.utcnow() + expires_delta
    else:
        expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)

    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt
```

Figure 10: JWT Creation and Expiry Handling

```python
class UserRegister(BaseModel):
    username: str
    password: str
    role: str

    @field_validator("password")
    @classmethod
    def password_complexity(cls, v: str) -> str:
        is_valid, error_msg = auth_utils.validate_password_strength(v)
        if not is_valid:
            raise ValueError(error_msg)
        return v

    @field_validator("username")
    @classmethod
    def username_format(cls, v: str, info) -> str:
        role = info.data.get("role")
        if not role:
            return v
        expected_start = role.lower()
        if not v.lower().startswith(expected_start) or not v.lower().endswith("@streeling.ac.uk"):
            raise ValueError(f"Username must follow the structure '{expected_start}...@streeling.ac.uk'")
        return v
```

Figure 11: Schema-Based Input Validation

Analysis: Figure 8 showcases the key security controls used by the program when inside the logic controller subsystem. Inside here, security controls such as schema based input validations (Figure 11) are used to ensure usernames and passwords meet the required strength criteria, preventing the most common reason for data breach entry [1]. Complementing this, controls such as the `@limiter.limit("5/minute")` (Figure 9) operation and session expiry (Figure 10) are also implemented, in order to prevent additional threats such as brute force attacks and session hijacking. Overall, this enforces a defense-in-depth approach, using layered defences to ensure compromise cannot happen due to a single security control failure.

5 Development & Challenges

During development, a logical challenge arose when it came to integrating the `/low-stock endpoint`, as shown in Figure 12.

```
@router.get("/low-stock")
def get_low_stock_books(db: Session = Depends(database.get_db),
                        current_user: models.User = Depends(get_current_user)):
```

Figure 12: /low-stock Vulnerability

This logic flaw created a broken access control, since only identity authentication was verified via `get_current_user` line, violating Least Privilege.

5.1 Solution

The fix involved creating a reusable `RoleChcker.py` class to intercept the request before it reached inventory logic to block students in the gateway, as shown in Figure 13.

```
class RoleChecker:
    def __init__(self, allowed_roles: list):
        self.allowed_roles = allowed_roles

    def __call__(self, user: models.User = Depends(get_current_user)):
        if user.role not in self.allowed_roles:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail=f"Role '{user.role}' is not authorized to access this resource. Required: {self.allowed_roles}"
            )
        return user
```

Figure 13: RoleChecker,py Class

5.2 Testing

This section will briefly confirm the working functionality of the previously mentioned implementations. It will highlight the test type, input, expected outcome and actual outcome, with evidenced output shown in Appendix C.

Table 1: Manual test cases validating functional behaviour and security enforcement

#	Test Type	Input	Expected Outcome	Pass Fail	Evidence
1	Functional	Student reserves available book	Reservation created; inventory updated	Pass	C.1
4	Functional	Librarian approves valid reservation	Loan activated; audit logged	Pass	C.1
5	Functional	Duplicate reservation by same user	Request rejected; no state change	Pass	C.2
3	Functional	Reservation for unavailable book	Request denied; inventory unchanged	Pass	C.3
2	Functional	Approval attempt with existing active loan	Approval rejected; state unchanged	Pass	C.4
6	Functional	Librarian denies reservation	Loan denied; reason recorded	Pass	C.5
7	Functional	User returns active loan	Loan closed; inventory restored	Pass	C.6
8	Security	Unauthenticated access to the system	Request blocked (unauthorised)	Pass	C.7
9	Security	Session exceeds configured inactivity timeout	Session invalidated; user required to re-authenticate	Pass	C.8
10	Security	Multiple failed login attempts within a short time window	Login requests blocked; source temporarily rate-limited	Pass	C.9

6 Critical Reflection and SSCoP Alignment

6.1 Mapping to SSCoP Principles

This section outlines the final implementation of the IMS, by evaluating it against the DSIT Software Security Code of Practice shown in Appendix D, with the results being displayed in Table 2.

Security Requirement and Feature	**Description**	**Mapped SSCoP Principle(s)**
Role-Based Access Control (RBAC)	Restricts access to inventory modification and sensitive endpoints based on user role, ensuring least privilege is enforced across the system.	1.4
Password Hashing (bcrypt)	Stores user credentials as salted password hashes rather than plaintext to reduce the impact of credential disclosure in the event of a database compromise.	1.1
Rate Limiting (Brute-force Protection)	Limits repeated authentication attempts on login endpoints to mitigate brute-force and credential stuffing attacks.	1.4
Session Management (JWT Expiry and Timeout)	Invalidates authentication tokens after inactivity or expiry to reduce the risk associated with unattended or long-lived sessions.	1.4
Input Sanitisation and Data Integrity Checks	Validates request data and enforces inventory state constraints (e.g., preventing overbooking) to maintain consistency and correctness of system data.	1.4
Audit Logging of Security-Relevant Events	Logs authentication attempts and privileged actions with user role, timestamp, and target reference to support traceability and forensic analysis.	1.1
Dependency Version Pinning (Supply Chain Security)	Pins third-party libraries to specific versions using a `requirements.txt` file to reduce the risk of malicious or breaking upstream dependency changes.	1.2
Secure Defaults via Global Security Headers	Applies hardened HTTP security headers (e.g., `X-Frame-Options`, `X-Content-Type-Options`) globally via middleware to ensure baseline protection by default.	1.4

Table 2: Mapping Security Requirements to the DSIT Software Security Code of Practice

6.2 Evaluation

This section will briefly outline the key strengths, limitations and areas for improvement noted while mapping the SSCoP Principles.

1.1 Secure development framework: Although the development process was methodological and followed a DevSecOps cycle, it failed to integrate formal industry standard frameworks such as ISO-aligned software standards [2], preventing the development from being formally standardised, but rather developer-driven. This limitation could be mitigated in the future by embedding structured risk assessment, ensuring the system is compliant and in line with formal industry standards.

1.2 Third-party risk: Despite the implementation being able to meet all the stated BDD requirements, it still managed to introduce third party risk. This is because development relied on third party libraries, which generally introduce more security flaws compared to first party code and also take longer to fix [3]. To address this issue, only highly trusted external libraries should be used, with regular updates applied to reduce exposure to evolving threats.

1.3 Testing before distribution: Even though during implementation manual testing was used to confirm basic UI and security processes, comprehensive automated test cases were not included. This made testing less efficient, since automation testing often uses frameworks that follow approaches, such as data-driven and keyword-driven testing, in order to enable faster execution and scalability [4]. To fix this, automated testing should be used for future iterations.

7 Conclusion

This report serves to evidence the implementation of the university's IMS system through a DevSecOps approach.

References

[1] Delinea. *Understanding the 2024 Verizon DBIR: Credential Compromise Dominates Cybersecurity Threats*. Accessed: January 2026. Delinea. 2024. URL: https://delinea.com/blog/2024-verizon-dbir-credential-compromise-dominates.

[2] International Organization for Standardization. *Software and IT Technologies*. Accessed: January 2026. ISO. n.d. URL: https://www.iso.org/sectors/it-technologies/software.

[3] Tim Anderson. *Third-party libraries cause more security woes than first-party code, open-source flaws take longer to fix*. Accessed: January 2026. DevClass. Mar. 2025. URL: https://devclass.com/2025/03/11/third-party-libraries-cause-more-security-woes-than-first-party-code-open-source-flaws-take-longer-to-fix/.

[4] GeeksforGeeks. *Manual Testing vs Automated Testing*. Last Updated: 10 Jan 2026. Accessed: January 2026. GeeksforGeeks. 2026. URL: https://www.geeksforgeeks.org/software-testing/software-engineering-differences-between-manual-and-automation-testing/.

Appendices

A Established System Requirements

Feature: Inventory Management System for Streeling University Library

As a library administrator,
I want to manage the inventory of books and resources efficiently,
So that I can ensure accurate tracking, availability, and secure access to library materials.

Scenario: Role-Based Access Control (RBAC)

Given different user roles (librarian and student),
When a user logs in,
Then the system should grant access based on their role,
And restrict unauthorised actions such as modifying inventory records or accessing audit logs.

Scenario: Define and manage inventory records

Given I am logged in as a librarian,
When I add, update, or remove a book from the inventory,
Then the system should reflect the changes,
And maintain a history of modifications for audit purposes.

Scenario: Audit logging of system activity

Given any user performs an action (e.g., login or inventory update),
When the action is completed,
Then the system should log the event with timestamp, user ID, and action details,
So that administrators can review system usage and detect anomalies.

B Low-Level Architectural Diagram

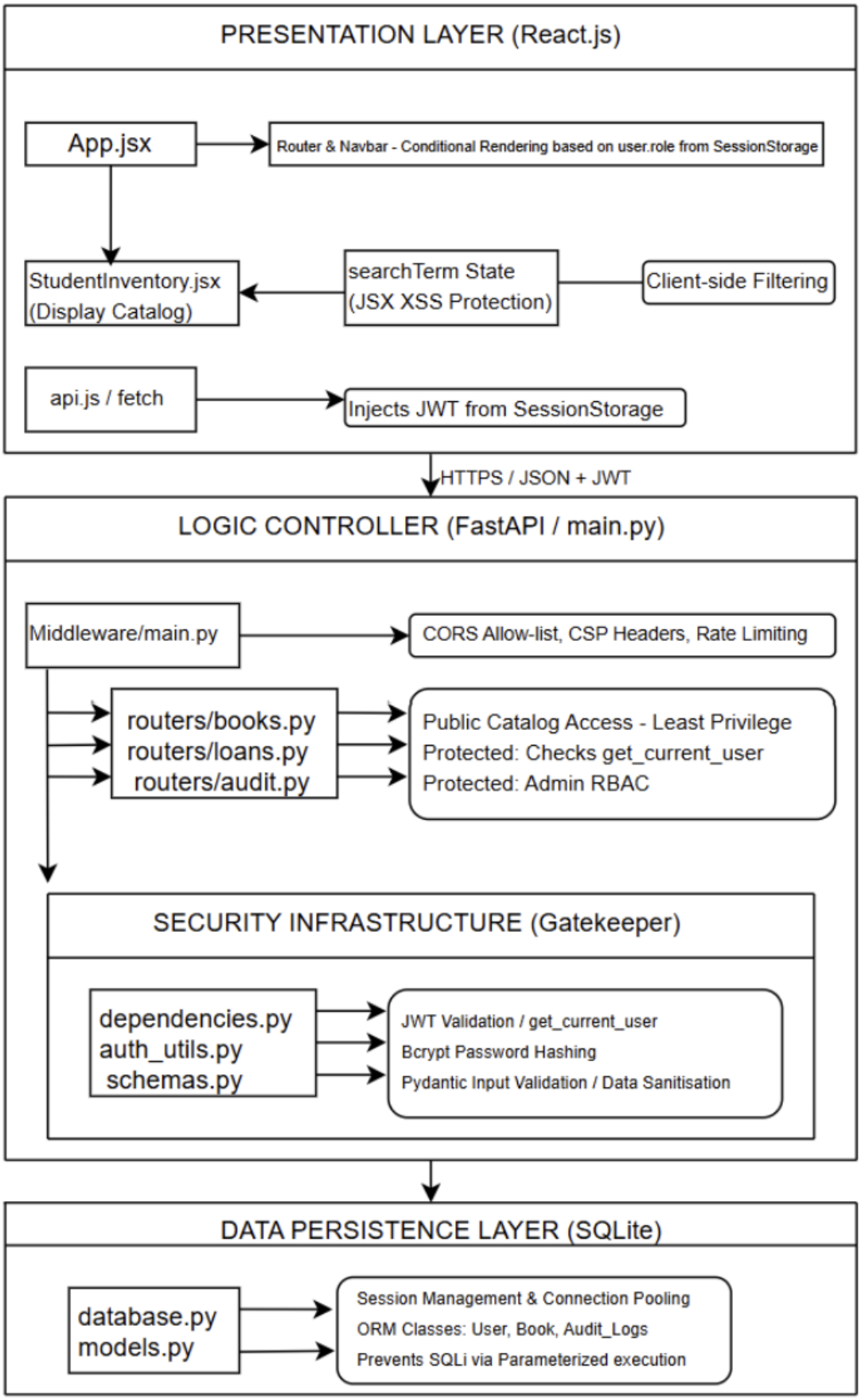


Figure 14: Three-Tier Low-Level Component Diagram

C Test Cases

C.1 Test 1 and 4: Student reserves available book

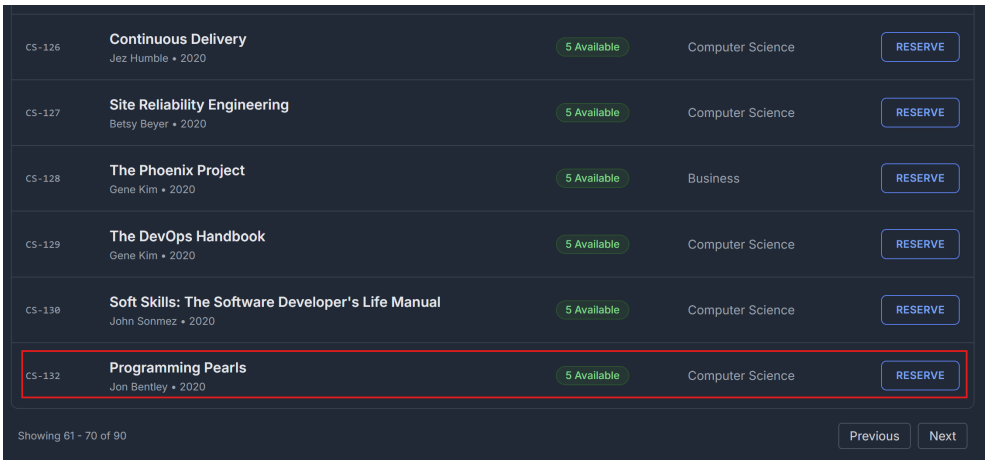


Figure 15: Student reserves available book

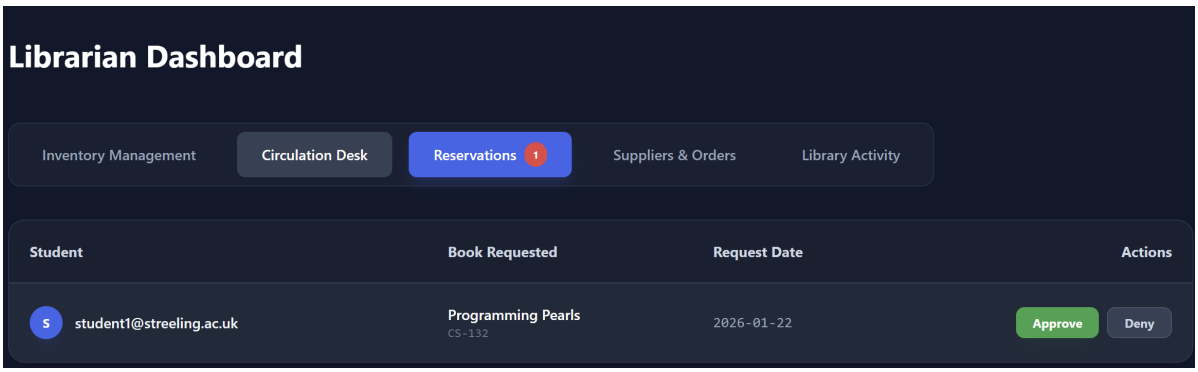


Figure 16: Reservation order comes to the librarian

Figure 17: Librarian Approves the Loan request

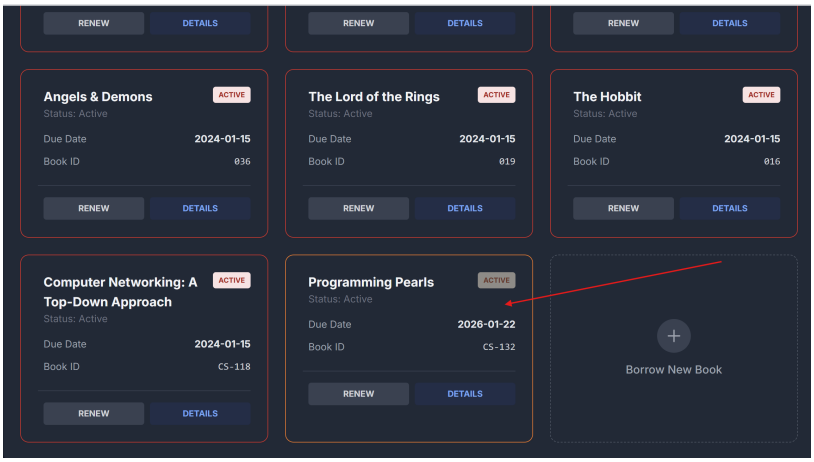


Figure 18: Reserved book shows up in the students dashboard

C.2 Test 2: Duplicate reservation by same user

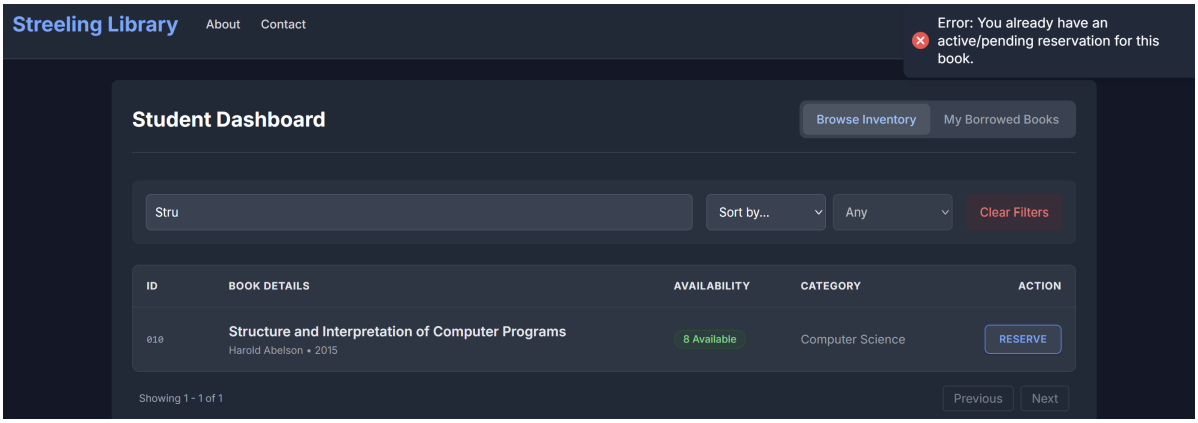


Figure 19: Request is denied for duplicate reservations

C.3 Test 3: Reservation for unavailable book

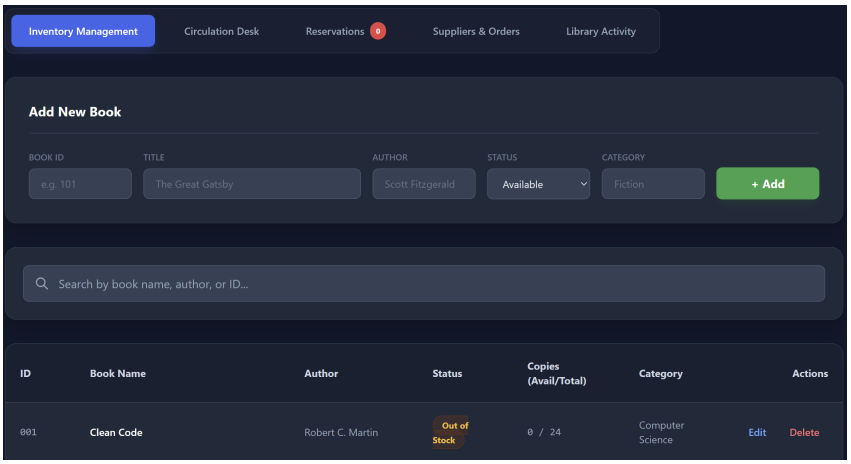


Figure 20: Book is shown to have 0 quantity

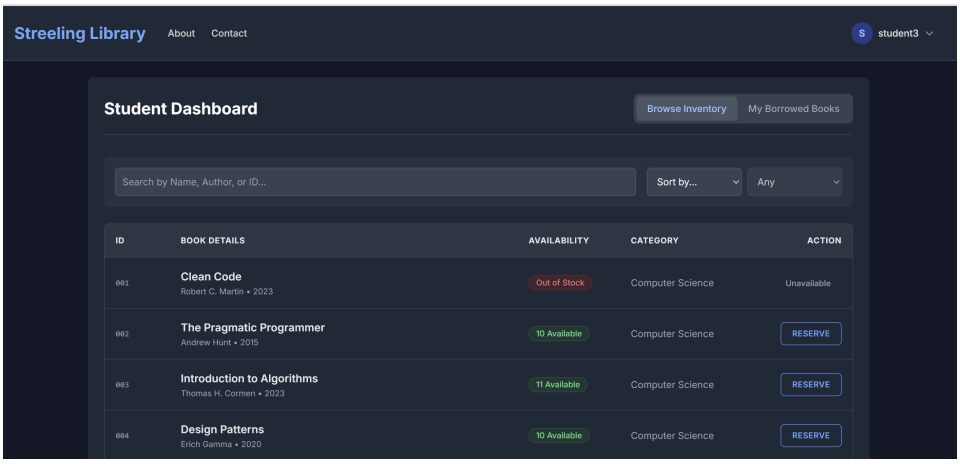


Figure 21: Book is unavailable to reserve

C.4 Test 5: Approval attempt with existing active loan

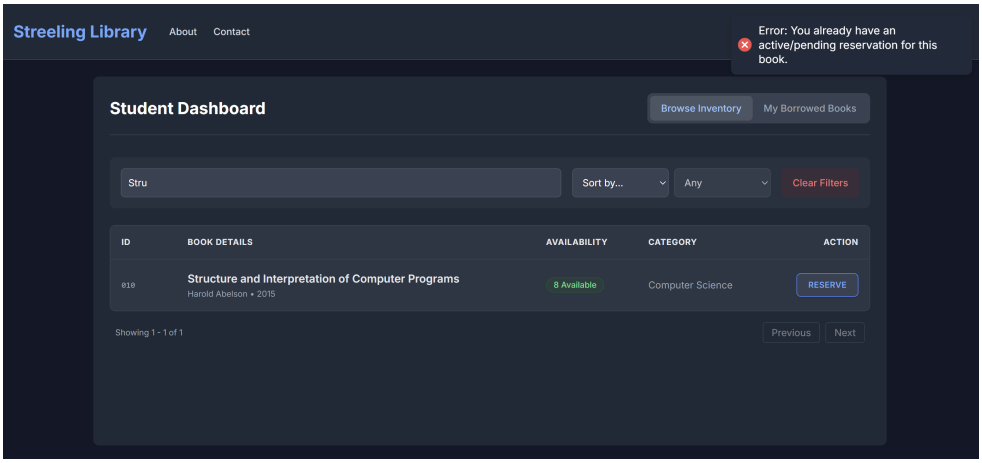


Figure 22: Double loan request is rejected

C.5 Test 6: Librarian denies reservation

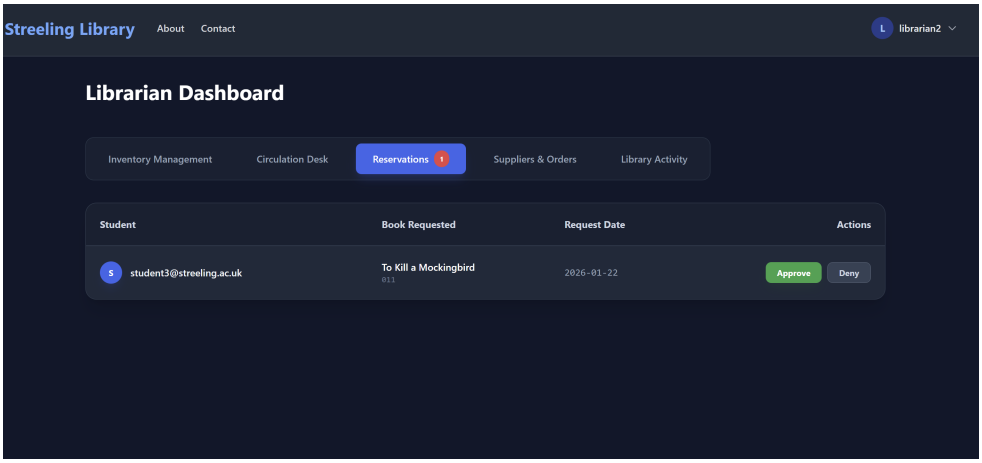


Figure 23: Reserved book is shown in librarian dashboard

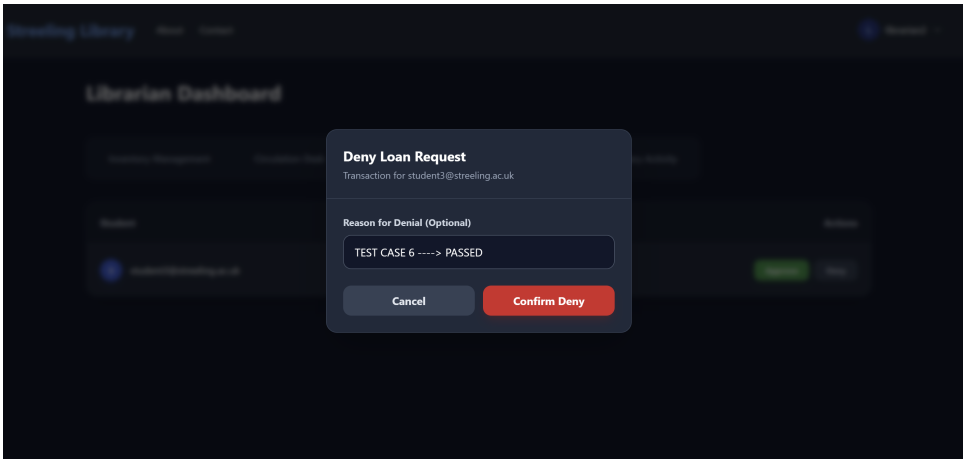


Figure 24: Loan request is denied

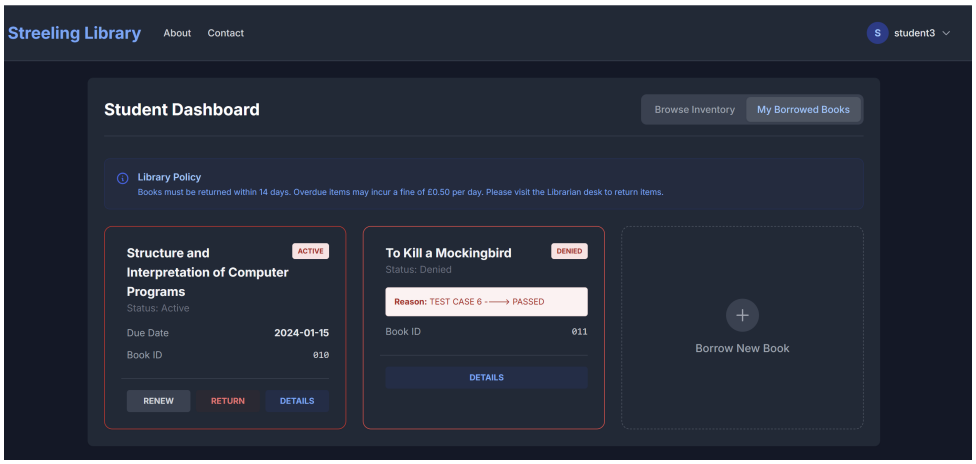


Figure 25: Denied result shown to the student

C.6 Test 7: User returns active loan

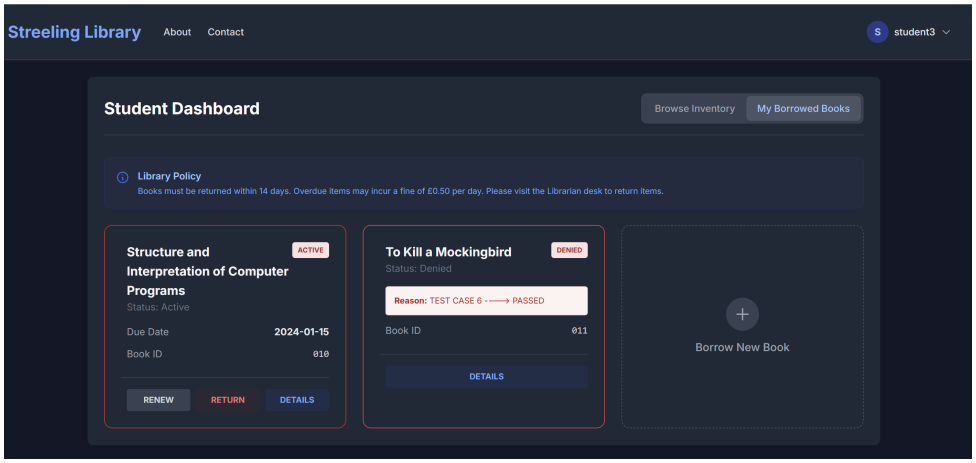


Figure 26: Showin active request of book that will be returned

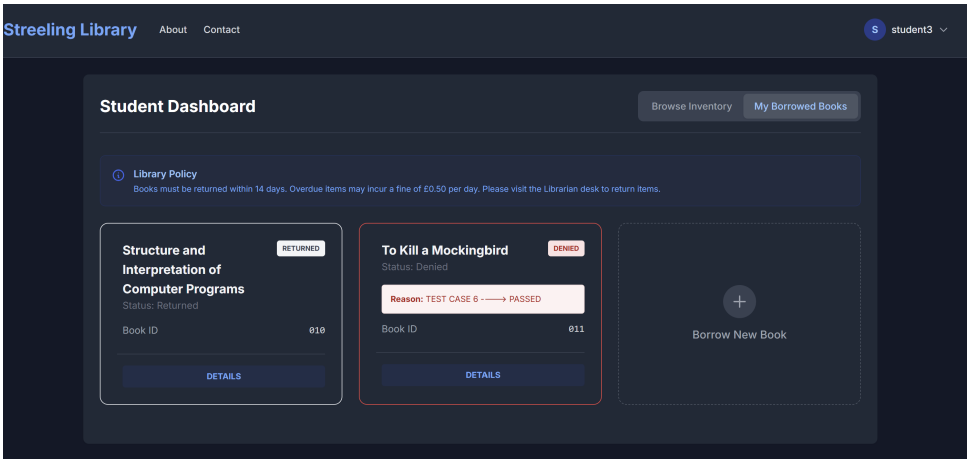


Figure 27: Book is returned

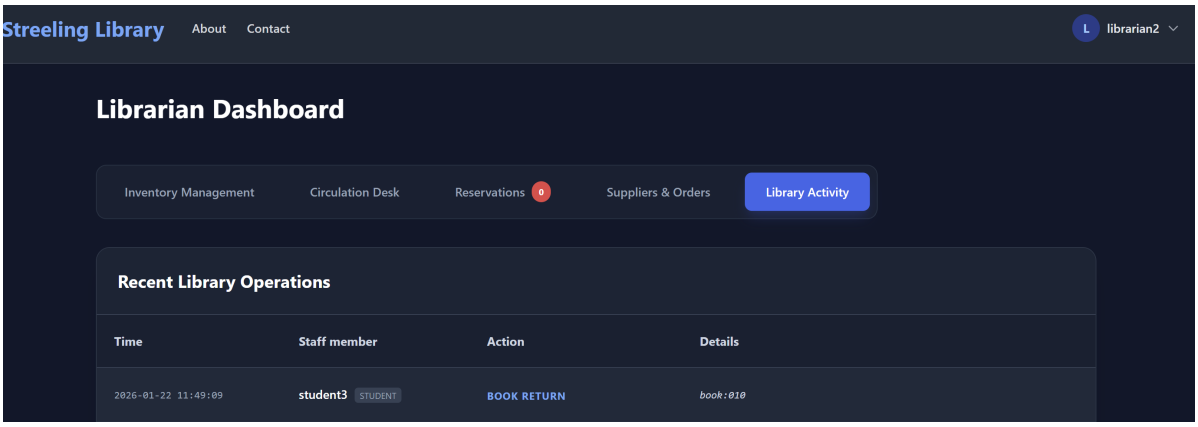


Figure 28: Return is audited

C.7 Test 8: Unauthenticated access to protected action

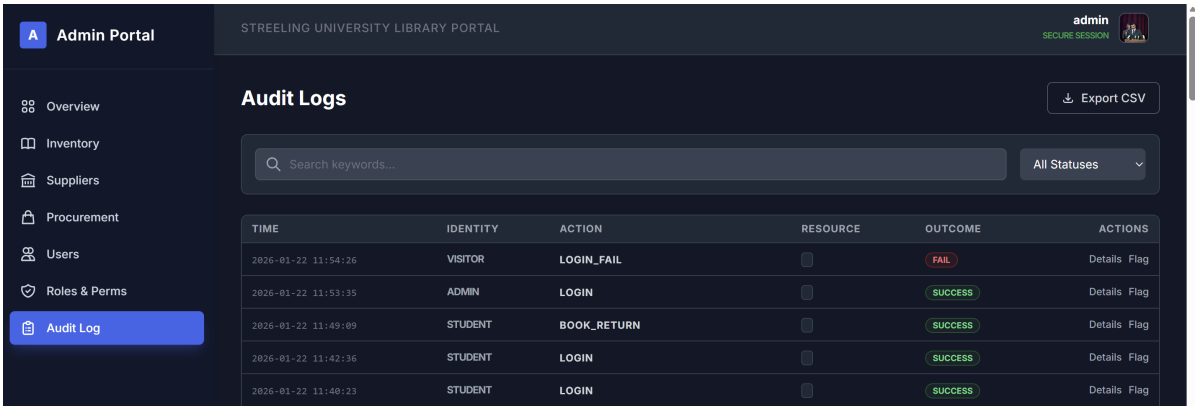


Figure 29: Failed login attempt audit

C.8 Test 9: Unauthenticated access to protected action

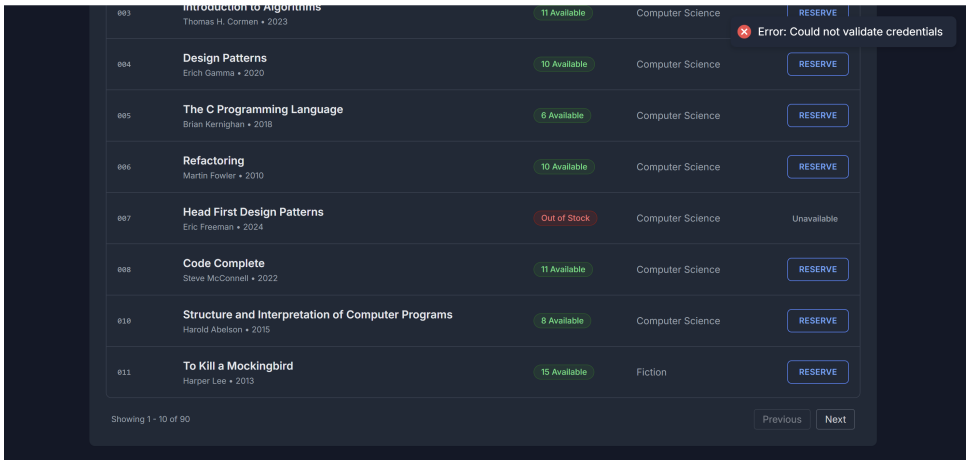


Figure 30: Session timed out

C.9 Test 10: Multiple failed login attempts within a short time window

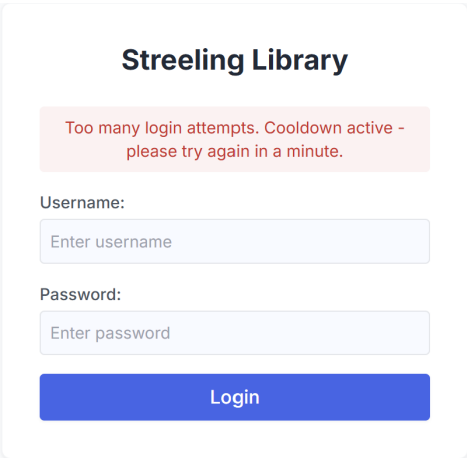


Figure 31: User requested to wait after multiple log-in attempts

D Software Security Code of Practice

1. Secure design and development

These principles ensure that the software is appropriately secure when provided.

The Senior Responsible Owner in vendor organisations shall gain assurance that their organisation achieves the following in relation to any software or software services sold by their organisation:

1.1 Follow an established secure development framework.
1.2 Understand the composition of the software and assess risks linked to the ingestion and maintenance of third-party components throughout the development lifecycle.
1.3 Have a clear process for testing software and software updates before distribution.
1.4 Follow secure by design and secure by default principles throughout the development lifecycle of the software.

2. Build environment security

These principles ensure that the appropriate steps are taken to minimise the risk of build environments becoming compromise and protect the integrity and quality of the software.

The Senior Responsible Owner in vendor organisations shall gain assurance that their organisation achieves the following in relation to any software or software services sold by their organisation:

2.1 Protect the build environment against unauthorised access.
2.2 Control and log changes to the build environment.

3. Secure deployment and maintenance

These principles ensure that the software remains secure throughout its lifetime, to minimise the likelihood and impact of vulnerabilities.

The Senior Responsible Owner in vendor organisations shall gain assurance that their organisation achieves the following in relation to any software or software services sold by their organisation:

3.1 Distribute software securely to customers.
3.2 Implement and publish an effective vulnerability disclosure process.
3.3 Have processes and documentation in place for proactively detecting, prioritising and managing vulnerabilities in software components.
3.4 Report vulnerabilities to relevant parties where appropriate.
3.5 Provide timely security updates, patches and notifications to customers.

4. Communication with customers

These principles ensure that vendor organisations provide sufficient information to customers to enable effective risk and incident management.

The Senior Responsible Owner in vendor organisations shall gain assurance that their organisation achieves the following in relation to any software or software services sold by their organisation:

4.1 Provide information to the customer specifying the level of support and maintenance provided for the software being sold.
4.2 Provides at least 1 year's notice to customers of when the software will no longer be supported or maintained by the vendor.
4.3 Make information available to customers about notable incidents that may cause significant impact to customer organisations.

Figure 32: DSIT Software Security Code of Practice principles (Sections 1–4)

E Codebase Access Information

The complete source code for the Inventory Management System (IMS) developed as part of this coursework is hosted on the University-provided GitLab platform, in accordance with the submission requirements.

Repository Access

- **Repository URL:** `https://git-lab.cyber.warwick.ac.uk/c5653630/ccse-ims-coursework1/-/tree/submission?ref_type=heads`
- **Access Type:** Private (University-authenticated access)
- **Branch Used for Submission:** `submission`

Supplementary Submission

Due to file size limitations imposed by the University submission system, the complete source code could not be submitted as a compressed (.zip) file via Tabula. The full codebase is therefore hosted on the University-provided GitLab platform as referenced above.

To ensure accessibility in the event of any issues with GitLab, a secure download link containing the complete codebase has been shared with the module team prior to submission.